# mini-contact-app

Ceci est le dossier d’analyse de l’application mini-contact-app, nom commercial All Your Contacts.

## Table of Contents


1 Introduction....2
1.1 Contexte....2
1.2 Remarque....2
2 Processus....2
3 Analyse fonctionnelle....2
3.1 Liste des acteurs....2
3.2 Use cases....3
3.3 Scénarios....3
3.3.1 UC crée un contact....3
3.3.2 Scénario UC consulte les contacts....5
3.3.3 Scénario UC consulte un contact....5
3.3.4 Scénario UC édite un contact....6
3.3.5 Scénario UC supprime un contact....6
3.3.6 Scénario UC exporte des contacts....7
3.3.7 Scénario UC importe des contacts....8
3.4 Schémas UI....10
3.4.1 Liste de contacts....10
3.4.2 Détail d’un contact....11
3.4.3 Formulaire de contact....13
3.4.4 Formulaire de contact....14
3.4.5 Contact Import....16
3.4.6 Import de contacts....17
3.5 Navigation....18
4 Analyse technique....18
4.1 Liste des contraintes....18
4.2 Design de la database....19
4.3 Design de l’application....20
4.4 Déploiement....21
5 Annexes....23
5.1 User stories, approche Kent Beck....23
5.1.1 US crée un contact....23
5.1.2 US consulte les contacts....23
5.1.3 US consulte un contact....24
5.1.4 US edite un contact....24
5.1.5 US supprime un contact....24
5.1.6 US exporte des contacts....24
5.1.7 US importe des contacts....25
5.2 User stories, approche commune....25
5.2.1 US crée un contact....25
5.2.2 US consulte les contacts....26
5.2.3 US consulte un contact....26
5.2.4 US edite un contact....27
5.2.5 US supprime un contact....27
5.2.6 US exporte des contacts....27

5.2.7 US importe des contacts....27
5.3 Classes entités....27
5.4 Glossaire....27
5.5 Workshop fonctionnalités....28

# 1 Introduction

mini-contact-app est une simple application de gestion des contacts.

## 1.1 Contexte

mini-contact-app est née du constat que nous avons besoin de gérer nos contacts.

Elle est développée par la communauté, est gratuite et le code est open source.

## 1.2 Remarque

C’est une application imaginaire. Le dossier d’analyse ci-présent a été rédigé pour servir d’exemple pour les élèves en bachelier en informatique de l’EPFC, Bruxelles, 2026.

C’est une application ultra-simple qui se résume à un CRUD de contact et l’import-export des contacts.

Elle contient des annexes qui sont des compléments :

- User stories, approche Kent Beck, cette approche remplace l’approche Use Case.
- User stories, approche connextra, idem.
- Classes entité : ce sont les classes qui font le lien avec la DB. Elles peuvent remplacer le schéma de la DB.
- Glossaire : toujours utile mais ici, il contient essentiellement des termes techniques qui font partie de la culture générale de l’informaticien. En général, le glossaire contiendra les termes du métier abordé par l’application.
- Workshop fonctionnalités est un petit exercice à mener en début de projet pour découvrir les UC / US.

# 2 Processus

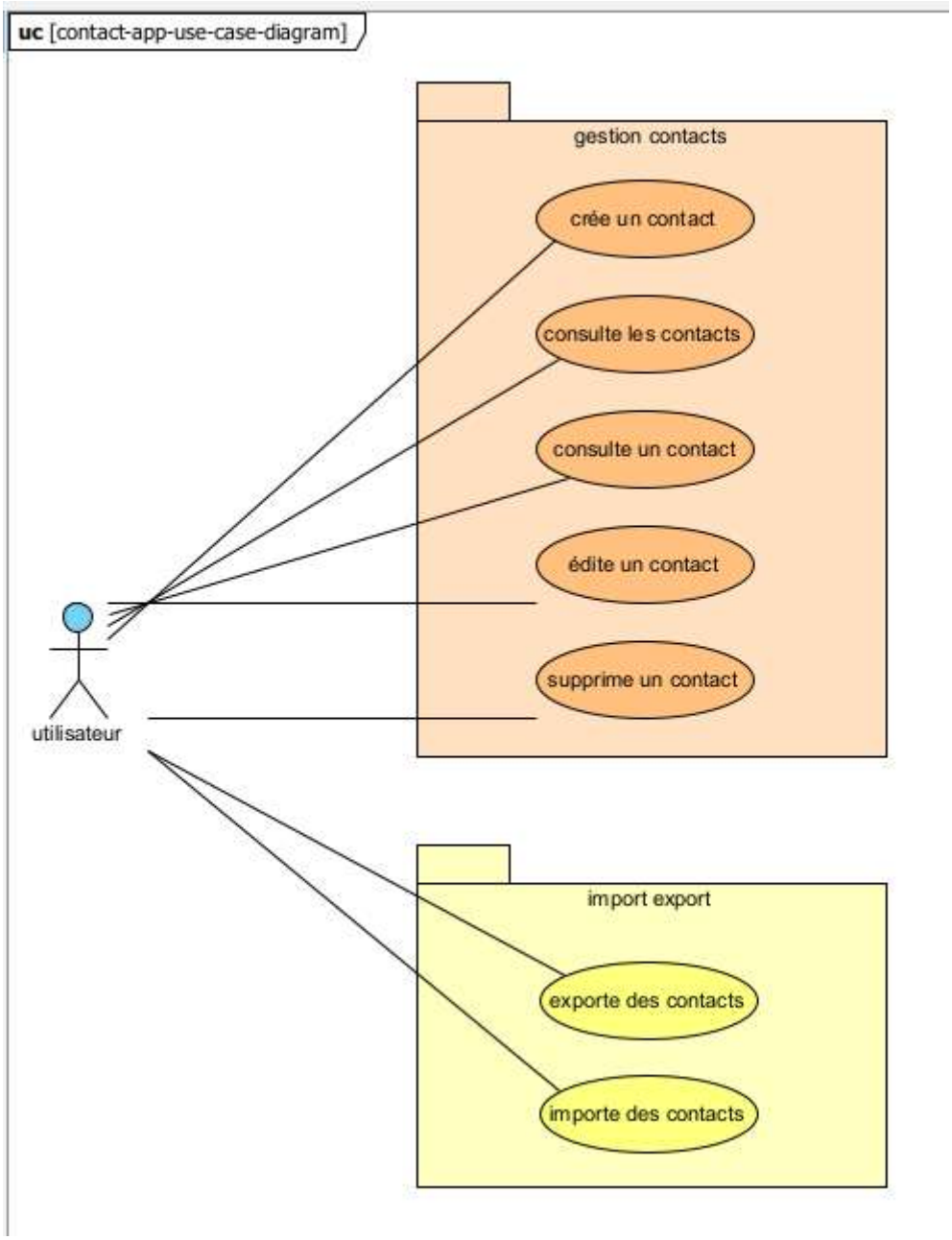
Il n’y a pas de processus.

# 3 Analyse fonctionnelle

## 3.1 Liste des acteurs

Utilisateur est le seul acteur. Il est l’utilisateur de mini-contact-app.

## 3.2 Use cases




## 3.3 Scénarios

Dans ce châpitre, nous documentons les use cases avec des scénarios.

### 3.3.1 UC crée un contact

#### *Description*

L'utilisateur crée un nouveau contact dans l'application.

### *Acteurs*

L’utilisateur

### *Préconditions*

Aucune

### *Scénario principal*

1. L’utilisateur complète le formulaire de création de contact
2. L’application vérifie que le formulaire est valide
   Si le formulaire n’est pas valide, l’application affiche un message d’erreur
   Si le formulaire est valide : l’application affiche un message de succès

   **Post conditions**
   - le contact est enregistré dans la DB (dans le cas d’un formulaire valide)

### *Scénario doublon possible*

1. Idem
2. Idem
3. Si un contact a déjà le même prénom et nom de famille ou
   si un contact a le même numéro de téléphone
   l’application affiche un popup indiquant :

   Un contact semblable existe déjà
   Le détail du contact (prénom, nom de famille …)
   Voulez-vous vraiment créer un nouveau contact ou voulez-vous éditer le contact existant?

4. Si l’utilisateur choisit la première option (créer un nouveau contact), l’application affiche un message de succès
   Si l’utilisateur choisir la seconde option (éditer le contact existant), l’application redirige sur le scénario “Editer un contact”.

   Post **conditions**
   - [dans le cas de la création d’un nouveau contact] le contact est créé dans la DB

### *Règles métiers*

- Un contact qui contient les informations suivantes
  - prénom (100 caractères maximum)
  - nom de famille (100 caractères maximum)
  - numéro de téléphone (20 caractères composés de chiffres, .- et espaces
  - adresse (200 caractères maximum, peut-être écrite sur plusieurs lignes)
  - notes (1000 caractères maximum, peut-être écrites sur plusieurs lignes)
- Un contact doit au minimum avoir un prénom ou un nom de famille ou un numéro téléphone

### 3.3.2 Scénario UC consulte les contacts

#### *Description*

L’utilisateur consulte la liste des contacts.

#### *Acteurs*

L’utilisateur

#### *Précondition*

Aucune

#### *Scénario principal*

1. L’utilisateur ouvre la page “contacts”
2. L’application affiche la liste des contacts sous forme tabulaire avec l’ensemble de l’information : Prénom, Nom de famille, Téléphone, Adresse, Notes
3. L’utilisateur peut appliquer un filtre global (filtre sur l’ensemble des champs)
4. L’application affiche la liste filtrée
5. L’utilisateur peut appliquer un filtre sur chaque champs
6. L’application affiche la liste filtrée

   **Post conditions**
   Aucune

#### *Règles métiers*

Aucune

### 3.3.3 Scénario UC consulte un contact

#### *Description*

L’utilisateur consulte un contact. C’est utile lorsque les notes sont particulièrement longues et donc lorsque le tableau des contacts n’affiche qu’une partie des notes.

#### *Acteurs*

L’utilisateur

#### *Précondition*

Aucune

#### *Scénario*

1. L’utilisateur sélectionne un contact dans la liste des contacts pour consultation
2. L’application affiche toutes les informations du contact

**Post conditions**
Aucune

### *Règles métiers*

Aucune

## 3.3.4 Scénario UC édite un contact

### *Description*

L’utilisateur édite un contact.

### *Acteurs*

L’utilisateur

### *Précondition*

Aucune

### *Scénario*

1. L’utilisateur sélectionne un contact dans la liste des contacts pour édition
2. L’application affiche le formulaire d’édition de contact rempli avec les données du contact choisi
3. L’utilisateur édite librement les champs
4. Le comportement est identique à l’UC crée un contact

**Post conditions**
Le contact modifié est enregistré dans la DB

### *Règles métiers*

Voir UC crée un contact

## 3.3.5 Scénario UC supprime un contact

### *Description*

L’utilisateur supprime un contact.

### *Acteurs*

L’utilisateur

### *Précondition*

Aucune

### *Scénario*

1. L’utilisateur sélectionne un contact dans la liste des contacts pour suppression
2. L’application affiche une demande de confirmation
3. Si l’utilisateur confirme, l’application affiche un message de succès indiquant que le contact a été supprimé

   **Post conditions**
   Le contact est supprimé de la DB

### *Règles métiers*

Aucune

## 3.3.6 Scénario UC exporte des contacts

### *Description*

L’utilisateur exporte des contacts.

### *Acteurs*

L’utilisateur

### *Précondition*

Aucune

### *Scénario*

1. L’utilisateur consulte les contacts, éventuellement il les filtre (voir scénario UC consulte les contacts)
2. L’utilisateur choisit d’exporter les contacts (éventuellement filtrés)
3. L’application affiche le formulaire d’export avec les informations suivantes :
   choix du format, deux valeurs possible : csv ou vCard
   choix du répertoire et nom du fichier (avec un composant UI File Picker)
4. L’utilisateur complète et confirme
5. L’application affiche l’avancement de l’export
6. L’application affiche un message de succès en cas réussite
   Sinon l’application affiche un message d’erreur

   **Post conditions**
   En cas de succès, le fichier existe et contient les contacts au format sélectionné.

### *Règles métiers*

Aucune

### 3.3.7 Scénario UC importe des contacts

#### *Description*

L’utilisateur importe des contacts.

#### *Acteurs*

L’utilisateur

#### *Précondition*

Aucune

#### *Scénario csv*

1. L’utilisateur choisit d’importer les contacts
2. L’application affiche le formulaire d’import des contacts. Il contient :

   un file picker permettant de choisir le fichier à importer
   ne permettant de sélectionner que des fichiers .csv ou .vcf
3. L’utilisateur sélectionne un fichier csv
4. Si le fichier est vide, l’application affiche un message d’erreur et l’utilisateur doit sélectionner un nouveau fichier
5. Sinon, l’application affiche le fichier sous forme tabulaire (elle interprète le csv)
6. L’application demande si la première ligne est une ligne d’en-tête
7. L’utilisateur complète oui ou non
8. L’application propose une correspondance entre les colonnes du csv et le contact de l’application, par exemple :
   colonne 1 + (valeur de la cellule 1) <===> prénom
   colonne 2 + (valeur de la cellule 2) <===> nom de famille
   colonne 3 + (valeur de la cellule 3) <===> téléphone
   colonne 4 + (valeur de la cellule 4) <===> adresse
   colonne 5 + (valeur de la cellule 5) <===> notes
9. L’utilisateur peut modifier la correspondance, ignorer une colonne, ignorer une donnée du contact
10. L’application indique les contacts qui sont potentiellement des doublons :
    si le nom et le prénom sont égaux à ceux d’un contact existant
    ou
    si le numéro de téléphone est égal à celui d’un contact existant
11. L’utilisateur peut demander d’ignorer certaines lignes
12. L’utilisateur demande d’importer le fichier
13. L’application importe le fichier
    et affiche les éventuelles erreurs

**Post conditions**
Les lignes importées sont de nouveaux contacts dans la DB

### *Scénario vCard*

1. Idem
2. Idem
3. L'utilisateur choisit un fichier .vcf
4. Si le fichier est vide, l'application affiche un message d'erreur
5. Sinon l'application effectue la correspondance entre le fichier vcf et l'application :

| Propriété vCard | Contact |
|---|---|
| N deuxième partie du champ<br>ex : *N:Rando;Renée* | FirstName<br>ex: Renée |
| N première partie du champ<br>ex : *N:Rando:Renée* | LastName<br>ex: *Rando* |
| TEL<br>ex: *TEL;TYPE=CELL:01234567* | Phone<br>ex: *01234567* |
| ADR<br>ex:<br>*ADR:;4;Rue des sirops;Bruxelles,1000,Belgique* | Address<br>ex:<br>*4 Rue des sirops; 1000 Bruxelles; Belgique* |
| NOTE | Notes |

   et affiche le résultat sous forme tabulaire

6. L'application signale les contacts qui sont potentiellement des doublons (cf scénario précédent)
7. L'utilisateur peut demander d'ignorer certaines lignes
8. L'utilisateur demande d'importer le fichier
9. L'application importe le fichier
   et affiche d'éventuelles erreurs

**Post conditions**
Les lignes importées sont de nouveaux contacts dans la DB

### *Règles métiers*

Voir UC "crée un contact".

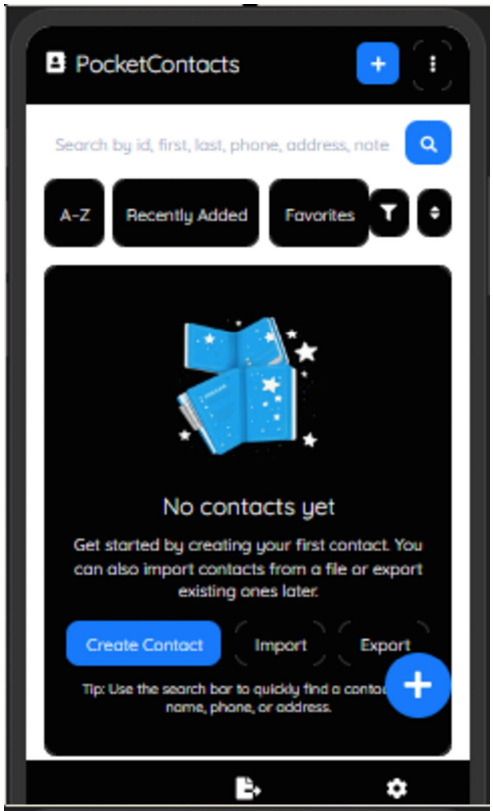
## 3.4 Schémas UI

Note : j'ai réalisé les schémas UI avec l'outil uizard qui fonctionne avec l'IA. C'est un résultat brut, je n'ai pas eu le temps d'explorer plus en détail l'outil.

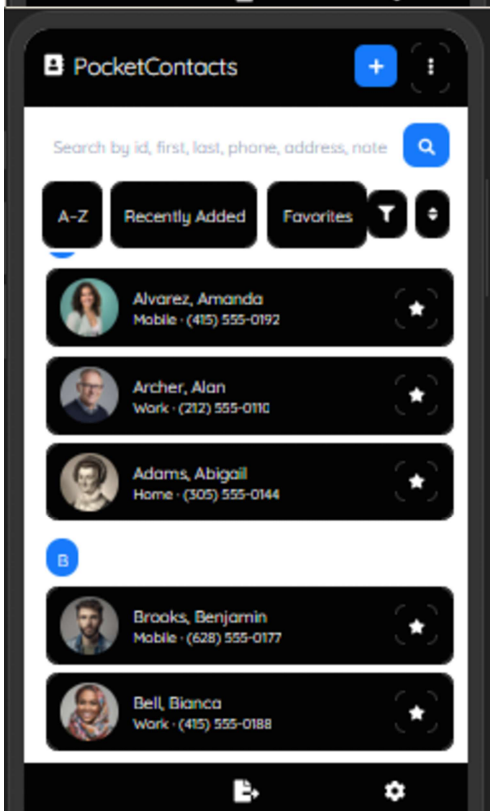
### 3.4.1 Liste de contacts

C'est l'écran de démarrage de l'application.

S'il n'y a pas de contacts, il affiche un message "No contacts yet" :




Lorsque des contacts ont été enregistrés, nous avons :



Les composants sont, de haut en bas :

- le bouton “+” pour ajouter un nouveau contact, il ouvre l’écran “Formulaire contact”.

- Le bouton pour ouvrir le menu (TBD est-ce nécessaire, quel menu ?)
- Le bouton “A-Z” pour filtrer sur une lettre de l’alphabet
- Le bouton “Recently Added” pour n’afficher que les contacts ajoutés récemment
- Le bouton “entonnoir” pour afficher le popup de paramètres de filtrage
- Le bouton “Favorite” pour n’afficher que les contacts favoris (avons-nous besoin de cette fonctionnalité ?)
- Le bouton pour inverser l’ordre de la l

- La carte contact contient les informations suivantes :
  - prénom & nom de famille
  - téléphone
- Un click sur un contact ouvre l’écran “Détail d’un contact”, prérempli, pour édition

### *Version desktop*

TODO

## 3.4.2 Détail d’un contact

Cet écran affiche le détail d’un contact. Il permet aussi d’accéder à l’édition d’un contact.

PocketContacts
Maya Alvarez
Senior Product Designer
Call
Message
Swipe ← Delete
Confirm Delete
Are you sure you want to permanently delete Maya Alvarez from PocketContacts? This action cannot be undone.
Cancel
Confirm Delete
Lastname
Alvarez
Firstname
Maya
Phone
+1 (415) 555-0198
Mobile • Tap to call
Address
482 Valencia St, Apt 4B, San Francisco, CA 94110
Tap to open in Maps
Notes
Met at the UX conference in Portland. Interested in collaborating on mobile redesign; prefers morning calls. Birthday: 1990-07-22. Tags: design, collaborator, priority
Edit
Delete
PocketContacts • v1.2.0

Les composants sont :

- “étoile” pour indiquer que le contact est un favori ou pas
- “call” pour effectuer un appel
- “message” pour envoyer un message
- Lastname : le nom de famille
- Firstname : le prénom
- Address : l’adresse avec une visualisation
- Notes : un espace de texte pour des notes
- “Edit” pour éditer le contacts
- “Delete” pour supprimer le contact
- “phone” pour effectuer un appel
- “message” pour envoyer un message
- le bouton pour activer / déactiver le contact

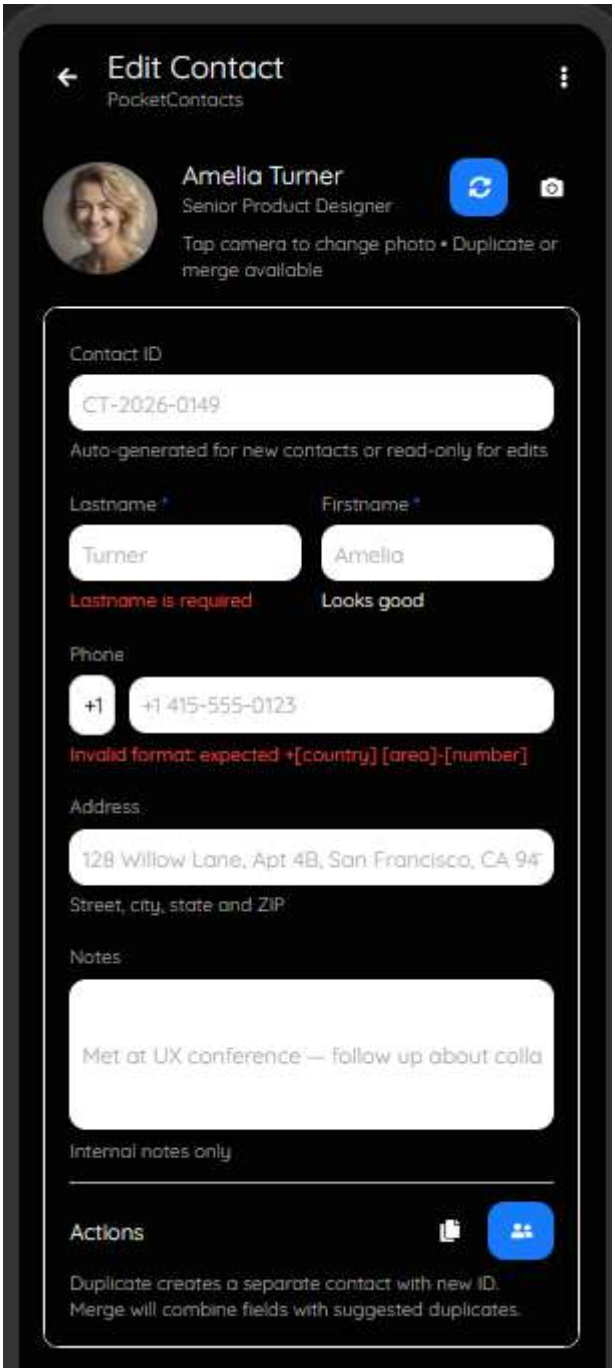
#### *Version desktop*

TODO

### 3.4.3 Formulaire de contact

#### *Version mobile*

Le formulaire de contact permet l’édition d’un contact existant ou la création d’un nouveau contact.




TODO : décrire les composants

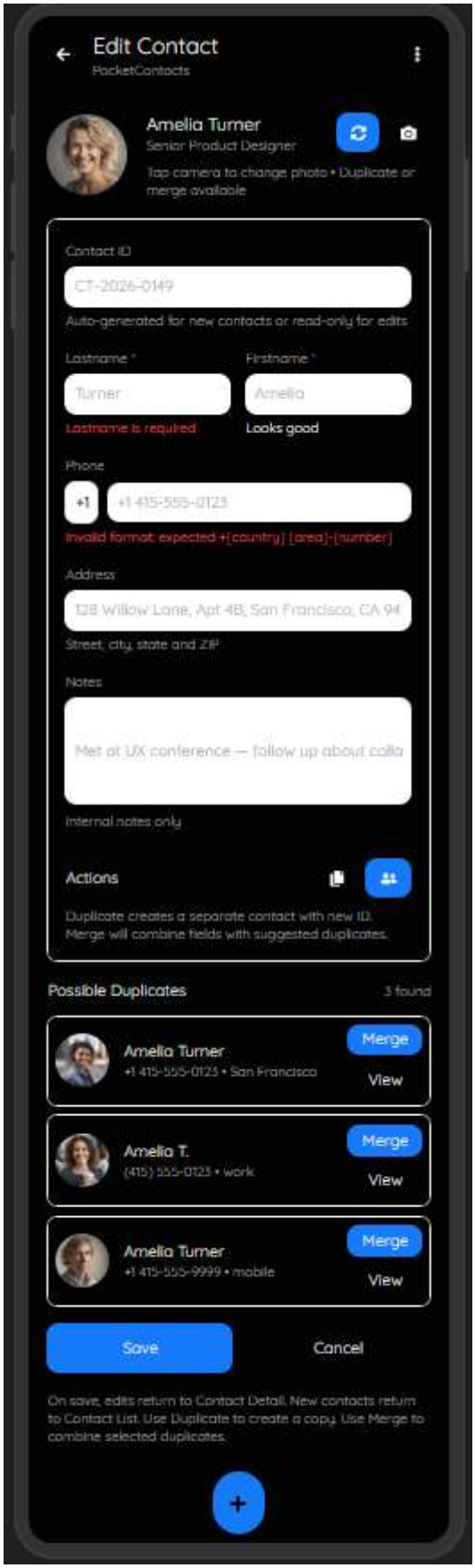
### *Version desktop*

TODO

## 3.4.4 Formulaire de contact

Le formulaire de contact est utilisé pour créer un nouveau contact et pour éditer un contact existant.

***Version mobile***



TODO expliquer les composants

### *Version desktop*

TODO

## 3.4.5 Contact Import

C’est l’écran d’import des contacts.

### *Version mobile*

TODO description des composants / actions

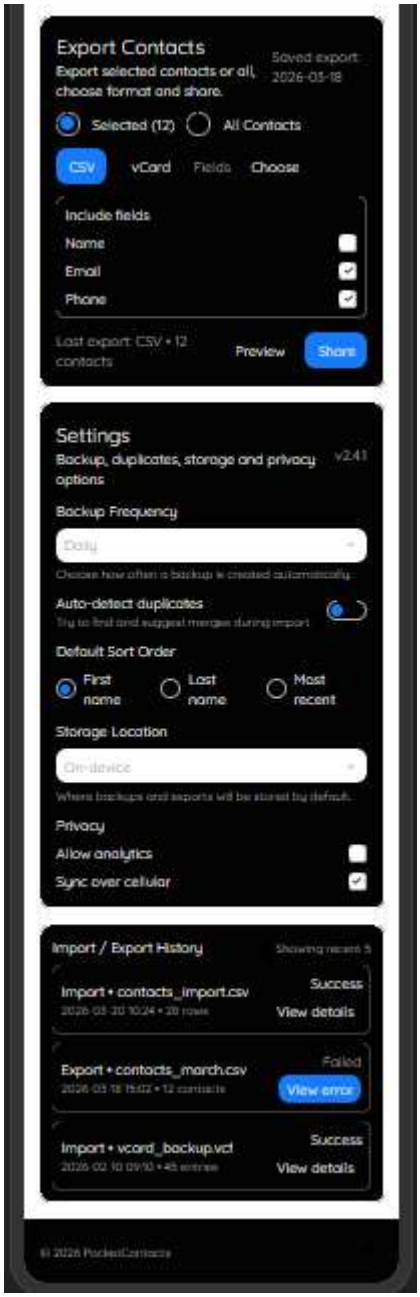
### *Version desktop*

TODO

### 3.4.6 Import de contacts

C’est l’écran pour les imports de contacts.

#### *Version mobile*

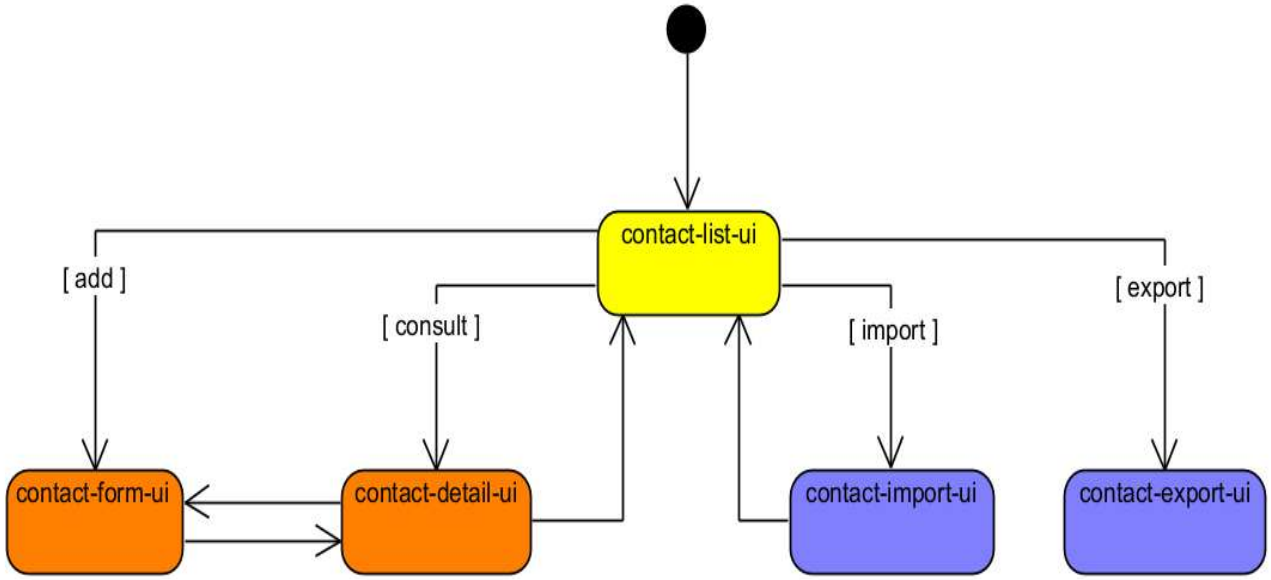



TODO description

***Version desktop***

TODO

## 3.5 Navigation




- contact-list-ui : écran d'accueil, affiche la liste de contacts
- contact-detail-ui : écran qui affiche le détail d'un contact
- contact-form-ui : formulaire de contact
- contact-import-ui : écran pour effectuer l'import de la liste des contacts
- contact-export-ui : écran pour effectuer l'export de la liste des contacts

# 4 Analyse technique

## 4.1 Liste des contraintes

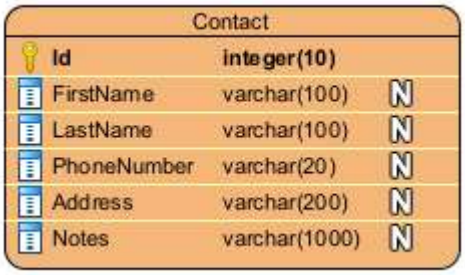
Les contraintes techniques sont :

1. L'application doit être installable sur Windows, en tant qu'exécutable avec les librairies et assets nécessaires.
2. L'application pour Windows sera fournie pour les architectures x86, x86-64 et arm.
3. L'application pour Windows sera fournie sous forme d'un zip par architecture processeur.
4. L'application doit être installable sur Linux, en tant qu'exécutable avec les librairies et assets nécessaires.

5. L’application pour Linux sera fournie pour les architectures x86, x86-64 et arm.
6. L’application pour Linux sera fournie sous forme d’un zip par architecture processeur.
7. L’application doit être installable sur MacOS, en tant que bundle .app empaquetté dans un .dmg.
8. L’application doit être installable sur iPhone/iPad en utilisant l’App Store.
9. L’application doit être installable sur Android en utilisant Play Store.
10. L’application doit être développée en dart.
11. L’application doit être basée sur le framework Flutter.
12. L’application doit utilisée une database Isar.
13. La database sqlite doit être embarquée dans l’application.
14. L’application doit avoir un temps de réponse inférieur à 1 seconde pour les UC :
    crée un contact
    consulte les contacts
    consulte un contact
    édite un contact
15. L’application doit avoir un temps de réponse inférieur à 1 seconde pour importer 1000 contacts (étape de correspondance et étape de sauvegarde dans la DB)
16. L’application doit avoir un temps de réponse inférieur à 1 seconde pour exporter 1000 contacts.
17. Les temps sont mesurés avec des ordinateurs modernes, de moins de 2 ans, moyen de gamme.

## 4.2 Design de la database

La database n’est composée que d’une seule table :

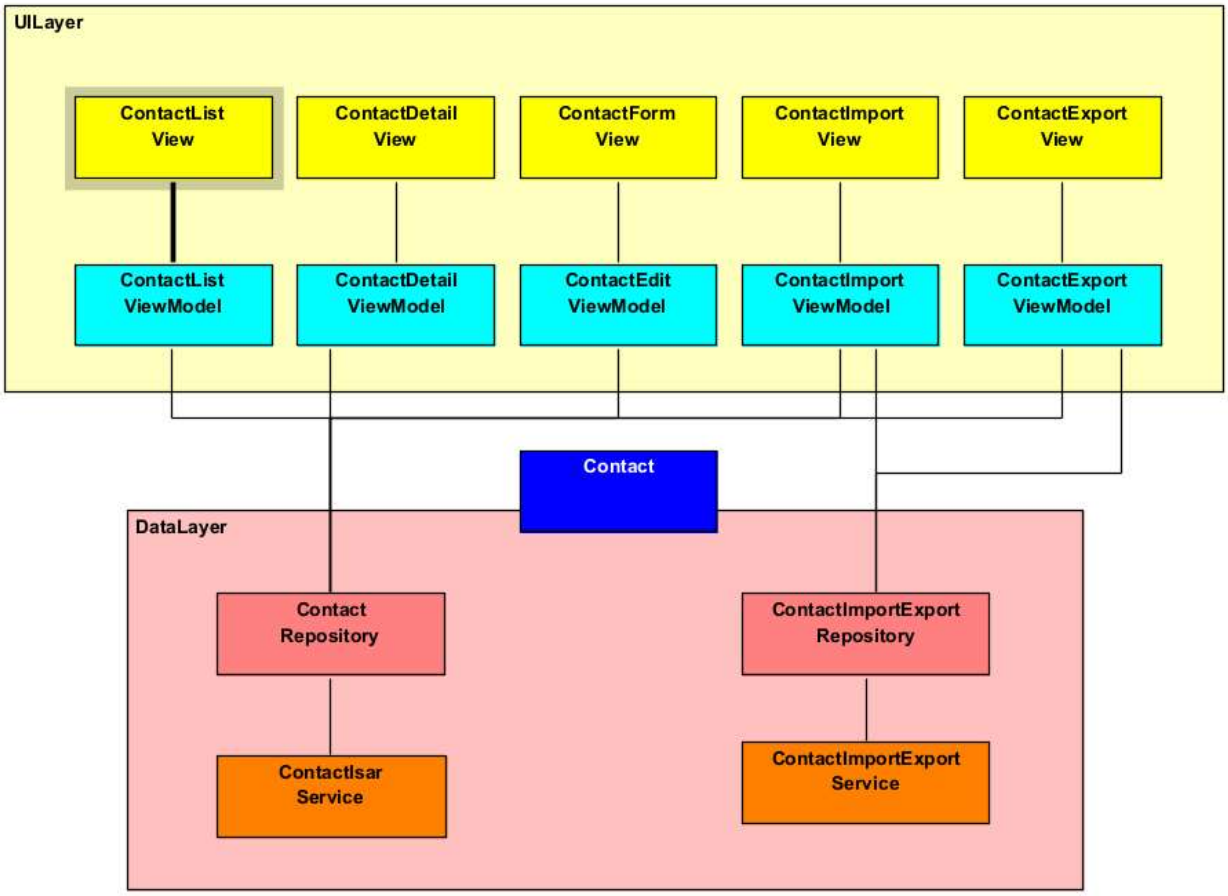


## 4.3 Design de l'application

L'application est codée en utilisant le design recommandé par l'équipe Flutter, https://docs.flutter.dev/app-architecture/guide.

Elle utilise le plugin pour sqlite https://pub.dev/packages/sqflite.

L'application sera composée de :

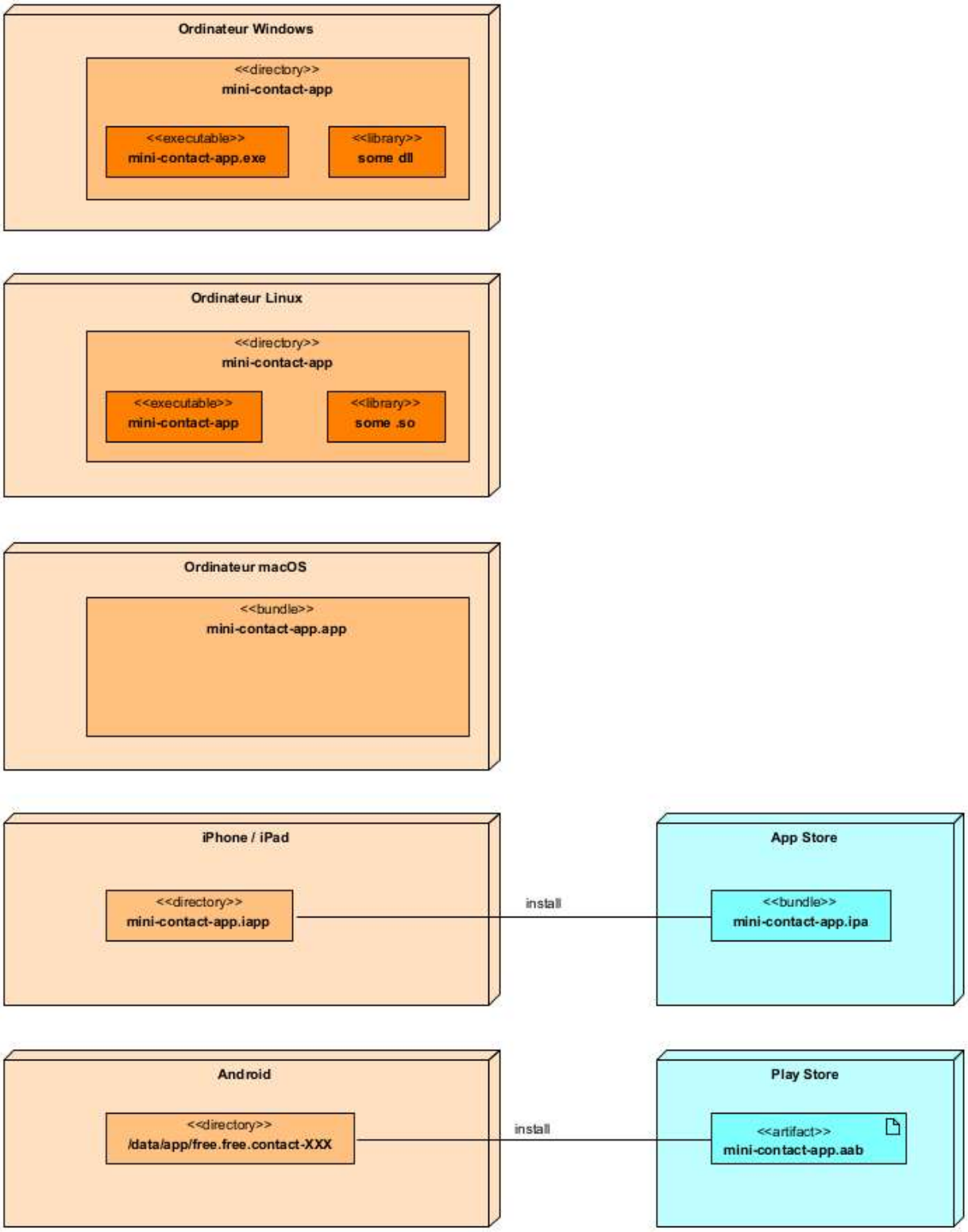



- UI layer est la couche de l'interface utilisateur (UI) :
  - ContactListView affiche la liste des contacts sous forme d'un tableau avec recherche et suppression.
  - ContactDetailView affiche le détail d'un contact
  - ContactEditView est le formulaire de création ou d'édition d'un contact.
  - ContactImportView est l'interface de configuration de l'import : choix du fichier, sélection des lignes, des colonnes …
  - ContactExportView est l'interface de configuration de l'export : choix du format, choix du fichier, ...
  - ContactListViewModel fournit à ContactListView la liste filtrée des contacts et expose les commandes de recherche et de suppression.

    - ContactDetailViewModel fournit à ContactDetailView les données du contact
    - ContactEditViewModel fournit à ContactEditView les données du contact à éditer et expose la commande de sauvegarde avec validation.
    - ContactImportViewModel fournit à ContactImportView l’état de l’import et expose les commandes d’import .
    - ContactExportViewModel fournit à ContactExportView l’état de l’export et expose les commandes
- Data layer est la couche de gestion des données :
    - ContactRepository est la source de vérité pour les données. Gère la gestion d’erreur et la logique mérier.
    - ContactIsarService contient le code pour communiquer avec la DB Isar, il s’interface au plugin sqflite.
    - ImpotrExportService s’occupe des imports et exports suivant différents formats : csv, vcard, file_picker.
- Contact est le modèle de données, c’est juste un mapping de la collection Contact de la DB Isar.

## 4.4 Déploiement

L’application doit être déployable sur des ordinateurs Windows, Linux et MacOS et sur des iPhone ou iPad et des Android :




Sur Windows, Linux et MacOS, nous installons manuellement l'application. L'utilisateur peut télécharger l'application sous le format d'un package zip à partir du serveur de free.free.

Sur iPhone/iPad et Android, nous installons le package sur App Store et Play Store. Ensuite, l'utilisateur installe lui même l'application.

# 5 Annexes

## 5.1 User stories, approche Kent Beck

### 5.1.1 US crée un contact

L'utilisateur crée un contact.
Les propriétés du contact sont :

- prénom : 100 caractères maximum
- nom de famille : 100 caractères maximum
- date de naissance : depuis 1900
-

#### *Critères d'acceptation*

- Vérifier qu'un Contact :
  - prénom: Sarah,
  - nom de famille :Vandamme,
  - téléphone : 025231010,
  - adresse : 3 Place du Centre, 1180 Uccle,
  - notes : ligne de note 1,\nligne de note 2,\nligne de note 3

    est créé.
- Vérifier qu'on ne peut pas enregistrer un contact ne contenant qu'une adresse et des notes.
- Vérifier qu'un message d'avertissement est affiché si un contact existant porte le même prénom et nom de famille et que l'application permet soit de créé le contact, soit d'éditer l'existant
- Vérifier qu'un message d'avertissement est affiché si un contact existant porte le même numéro de téléphone et que l'application permet soit de créé le contact, soit d'éditer l'existant
- Vérifier qu'un message d'avertissement est affiché si une des propriétés dépasse le nombre de caractères autorisé

### 5.1.2 US consulte les contacts

TODO

### 5.1.3 US consulte un contact

TODO

### 5.1.4 US edite un contact

TODO

### 5.1.5 US supprime un contact

TODO

### 5.1.6 US exporte des contacts

TODO

### 5.1.7 US importe des contacts

TODO

## 5.2 User stories, approche commune

### 5.2.1 US crée un contact

En tant qu'utilisateur, je veux pouvoir créer un contact ainsi je peux le retrouver facilement.

#### *Critères d'acceptation*

1. L'utilisateur doit pouvoir saisir un contact ne contenant que le prénom
2. L'utilisateur doit pouvoir saisir un contact ne contenant que le nom de famille
3. L'utilisateur doit pouvoir saisir un contact ne contenant que le numéro de téléphone

4. Le numéro de téléphone ne peut contenir que des chiffres et les caractères +,  espace et -
5. Le champs adresse doit être multiligne
6. Si un contact existant a le même prénom et nom de famille, l'application doit afficher un message d'information et permettre

   soit de sauver le nouveau contact

   soit passer à l'édition du contact existant
7. Si un contact existant a le même numéro de téléphone, l'application doit afficher un message d'information et permettre
   soit de sauver le nouveau contact
   soit passer à l'édition du contact existant
8. Une fois le bouton enregistrer cliqué, le contact doit être ajouté à la DB
9. Un message de confirmation doit être affiché après l'enregistrement en DB.

### 5.2.2 US consulte les contacts

En tant qu'utilisateur, je veux pouvoir consulter les contacts ainsi je peux trouver le contact ou les contacts que je cherche.

#### *Critères d'acceptation*

1. L'application doit afficher la liste des contacts de manière tabulaire avec toutes leurs propriétés excepté l'id.
2. L'application affichera une ligne par contact.
3. La propriété "notes" sera tronquée si nécessaire.
4. L'utilisateur doit pouvoir faire une recherche textuelle globale.
5. L'utilisateur doit pouvoir faire une recherche par propriété.
6. L'application doit afficher les contacts en moins d'une seconde.

### 5.2.3 US consulte un contact

En tant qu'utilisateur, je veux pouvoir consulter un contact ainsi je peux lire l'ensemble de la proprités notes.

#### *Critères d'acceptation*

1. L'utilisateur pourra sélectionner un contact dans la liste des contacts afin d'afficher l'information de ce contact.
2. Le contact sera affiché de manière semblable au formulaire de contact utilisé dans la US crée un contact, excepté que le formulaire ne sera pas éditable.
3. La propriété Notes sera affichée complètement.

4. L’utilisateur aura la possibilité de passer en mode édition (=> voir US édite un contact).

### 5.2.4 US edite un contact

TODO

### 5.2.5 US supprime un contact

TODO

### 5.2.6 US exporte des contacts

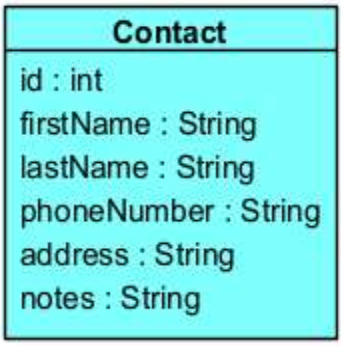
TODO

### 5.2.7 US importe des contacts

TODO

## 5.3 Classes entités

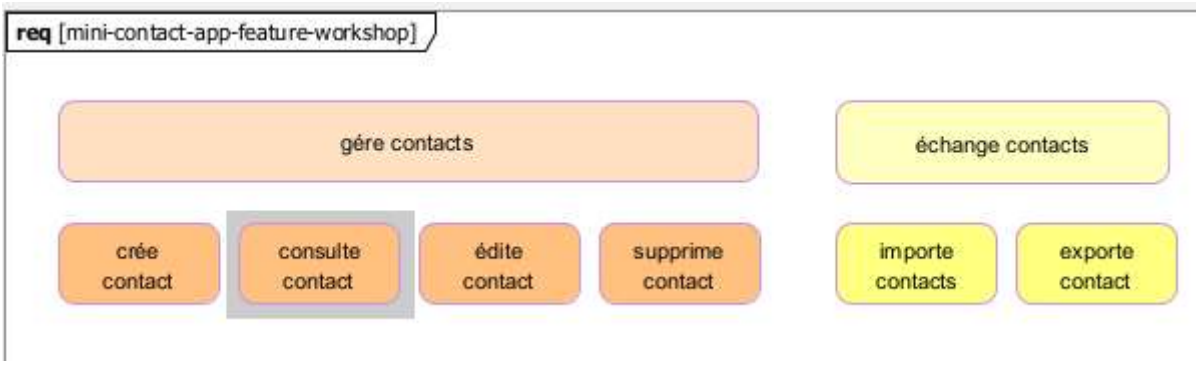
Le diagramme des classes entités est :




## 5.4 Glossaire

| Terme | Signification |
|---|---|
| CRUD | Create Read Update Delete (les opérations de base en DB) |
| csv | Comma separated value est un format de fichier pour enregistrer les données tabulaires.<br>Wikipedia |
| UC | Use Case |

| | |
|---|---|
| US | User Story |
| vCard | Format d'échange pour des données personnelles<br>wikipedia, rfc6350 |
| VCF | Extension des fichiers vCard |

## 5.5 Workshop fonctionnalités

Pendant un workshop, nous avons chercher les fonctionnalités de l'application et les avons regroupées.